Лабораторна робота №1

Тема: Створення монолітного CRUD застосунку з шаровою архітектурою

В цій лабораторній роботі вам слід почати спільну командну роботу в GitHub та зробити простий монолітний CRUD-застосунок, згідно з шаровою архітектурою. CRUD - означає виконання базових операцій Create, Read, Update, Delete з даними.

Доменну область можете обрати свою (приклади буде надано далі).
Справжню базу даних в цій роботі використовувати не обов'язково - можна використати in-memory сховище (просто реалізувати збереження в коді).

Потрібно зробити лише бекенд-сервер - застосунок, взаємодіяти з яким можна буде через REST API. Іншими словами, верхнім шаром архітектури повинен бути контролер, який дозволить взаємодію із застосунком за допомогою HTTP та надасть REST - ендпойнти.

Теоретичні відомості до роботи можете послухати/подивитися в Лекції 3.

Стек мов/технологій обирайте за бажанням.

Мета роботи

Навчитися організовувати структуру програмного застосунку за принципом **шарової архітектури (Layered Architecture)** та забезпечувати коректні залежності між компонентами системи.

Завдання

Розробити **монолітний програмний застосунок**, використовуючи принципи **шарової архітектури**.

Як приклад доменної області можна використати одну з систем (або інші):

- система керування задачами (Task Manager)
- система обліку книг у бібліотеці
- система керування замовленнями

Застосунок повинен підтримувати базові операції **CRUD** (Create, Read, Update, Delete).

Вимоги до архітектури

Проєкт повинен містити такі шари:

Controller

- приймає запити
- викликає методи сервісного шару
- повертає результат

Service

- реалізує бізнес-логіку
- викликає репозиторії

Repository

- забезпечує доступ до даних
- працює з базою даних або внутрішнім сховищем

Model

- описує структуру сутностей системи

Архітектурні обмеження

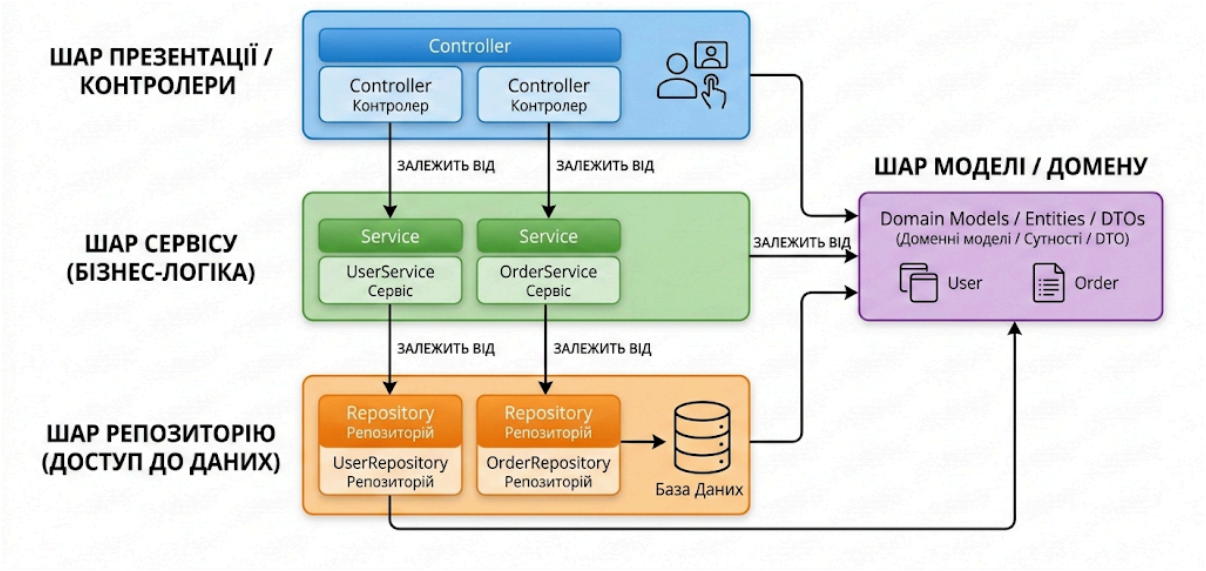
Забороняється:

- виклик Repository із Controller
- доступ Controller безпосередньо до бази даних
- розміщення бізнес-логіки у Controller

Правильний порядок викликів:

Controller → Service → Repository

Model, в цій лабораторній роботі, буде по суті sidecar - спільним модулем, що не належить до жодного з трьох згаданих шарів, а controller, service та repository можуть залежати від нього.



Очікувана структура проєкту

Приклад для системи керування задачами (Task Manager)

```
src
 ├── controller
 │    TaskController
 │
 ├── service
 │    TaskService
 │
 ├── repository
 │    TaskRepository
 │
 └── model
      Task
```

Ендпойнти контролера (приклад)

Контролер повинен приймати HTTP-запити відповідно до REST API, наприклад:

1. **GET /tasks**
   Повертає список усіх задач.
2. **GET /tasks/{id}**
   Повертає задачу за ідентифікатором.
3. **POST /tasks**
   Створює нову задачу.

4. **PUT /tasks/{id}**
   Оновлює існуючу задачу.
5. **DELETE /tasks/{id}**
   Видаляє задачу.

Перевірка ендпойнтів - через Postman, curl, або Swagger.

Вимоги:

- Повернення коректних HTTP-статусів (200, 201, 400, 404).
- Обробка помилок (наприклад, якщо задача не знайдена).
- Дані передаються у форматі JSON.

Предметна область (приклад)

Приклад моделі для системи управління задачами (система що відповідає за створення, перегляд, оновлення, видалення задач):

Інтерфейс для проекту Task Manager. Сутність: Task

Обов'язкові поля:

- id (унікальний ідентифікатор)
- title (назва задачі)
- description (опис)
- status (наприклад: NEW / IN_PROGRESS / DONE)
- createdAt (дата створення)

Моделі даних робіть відповідно до обраної предметної області.

Що потрібно зробити

1. Налагодити командну роботу (або особисту, якщо ви один), у спільному GitHub-репозиторії.
2. Створити як мінімум одну основну сутність системи
   (наприклад: Task, Book, Order).
3. Реалізувати операції:
   - створення об'єкта
   - отримання списку об'єктів
   - редагування
   - видалення
3. Організувати код відповідно до шарової архітектури, виконавши вимоги вище.
4. Зробити звіт.
5. Захистити роботу - показати роботу системи, протестувати ендпойнти та вивід, пояснити роботу коду та відповісти на питання викладача.

Зміст звіту

У звіті слід подати:

1. Назву та мету роботи
2. Короткий опис обраної предметної області
3. Структуру проєкту
4. Просту діаграму архітектури
5. Короткий опис реалізованих компонентів
6. Посилання на GitHub репозиторій проекта
7. Висновки